

Sistemas Distribuídos — Aula 26

Sistemas Operacionais Distribuídos: Sistemas de Arquivos Distribuídos · 12/11 · Prof. Leandro Borges

Fechando o bloco de sistemas operacionais distribuídos

O bloco de sistemas operacionais distribuídos (SODs), aberto na Aula 24, seguiu um percurso deliberado. Naquela aula construímos o vocabulário básico: o que distingue um SOD de um simples Sistema Operacional de Rede, e quais modelos de arquitetura — estação de trabalho, banco de processadores, híbrido — organizam os processadores disponíveis em um sistema desse tipo. Na Aula 25, aplicamos esse vocabulário a um problema concreto: como um SOD decide onde cada processo executa, por meio de gerência de processos e escalonamento distribuído, incluindo migração de processos e balanceamento de carga entre os nós do sistema.

Esta aula fecha o bloco tratando do outro recurso clássico que um sistema operacional gerencia, além de processos: os arquivos. A pergunta que orienta a aula de hoje é direta — se processos podem migrar entre máquinas e ser escalonados de forma transparente, o que acontece com os arquivos que esses processos leem e escrevem? Onde eles ficam fisicamente armazenados, e como vários processos, rodando em máquinas diferentes, conseguem compartilhá-los sem que cada um precise saber os detalhes de onde e como os dados estão guardados? A resposta é o assunto central desta aula: os sistemas de arquivos distribuídos.

O que é um sistema de arquivos distribuído

Um sistema de arquivos distribuído, ou DFS (do inglês *distributed file system*), é um serviço que permite que processos em execução em máquinas diferentes acessem e compartilhem arquivos como se estivessem todos operando sobre um único sistema de arquivos local. Do ponto de vista de quem programa ou usa o sistema, existe uma hierarquia de diretórios coerente e única — um único espaço de nomes — mesmo que, por trás dela, os arquivos estejam fisicamente distribuídos por vários servidores diferentes, cada um responsável por uma parte dos dados.

Essa definição retoma diretamente a ideia de transparência que atravessa toda a disciplina desde a Semana 2, e que a Aula 24 já havia aplicado ao sistema operacional como um todo. Um DFS é a aplicação dessa mesma ideia especificamente ao armazenamento: o objetivo é que um cliente nunca precise saber, para ler ou escrever um arquivo, em qual máquina física aquele arquivo está guardado, nem precise mudar a forma como acessa o arquivo caso ele seja movido de um servidor para outro. Construir esse tipo de ilusão de forma confiável, sobre uma rede que pode ser lenta, sujeita a falhas e compartilhada por múltiplos clientes simultâneos, é um problema de projeto bem mais delicado do que pode parecer à primeira vista — e é o que motiva os requisitos discutidos a seguir.

Requisitos centrais: transparência e semântica de consistência

Dois requisitos, em particular, costumam ser tratados como centrais em qualquer DFS. O primeiro é a transparência de acesso e de localização. Transparência de acesso significa que as mesmas operações servem para ler ou escrever um arquivo, seja ele local ou remoto — o programa cliente não precisa usar uma API diferente, nem saber de antemão se está lidando com um arquivo próximo ou distante. Transparência de localização, por sua vez, significa que o nome de um arquivo não revela onde ele está fisicamente armazenado, e que o arquivo pode, em princípio, ser movido de um servidor para outro sem que o caminho usado pelo cliente para acessá-lo precise mudar.

O segundo requisito central é a semântica de consistência: o que exatamente um cliente deve enxergar quando outro cliente, em outra máquina, está lendo ou escrevendo o mesmo arquivo ao mesmo tempo. Em um sistema de arquivos local, a resposta é simples e é conhecida como semântica Unix — toda escrita é imediatamente visível para qualquer leitura seguinte, porque existe um único kernel controlando todo o acesso ao disco. Em um sistema distribuído, garantir essa mesma visibilidade imediata entre clientes diferentes, cada um potencialmente com uma cópia em cache do arquivo, tem um custo alto em coordenação e tráfego de rede. Por isso, boa parte dos DFS reais relaxa deliberadamente essa garantia, adotando semânticas mais fracas — como veremos ao estudar o NFS clássico — em troca de desempenho e simplicidade.

Estudo de caso clássico: o NFS

O Network File System (NFS), desenvolvido pela Sun Microsystems na década de 1980, é o exemplo de referência de um DFS organizado em torno do modelo cliente-servidor mais simples possível. Um servidor NFS exporta um ou mais diretórios de sua árvore local; um cliente monta esse diretório exportado em algum ponto de sua própria árvore de diretórios, e a partir daí os arquivos daquele diretório remoto passam a ser acessados pelo cliente como se fossem locais. A comunicação entre cliente e servidor acontece por chamadas de procedimento remoto (RPC), tradicionalmente sobre os protocolos de transporte UDP ou TCP.

Duas características de projeto do NFS clássico merecem destaque. A primeira é que, nas suas versões originais (NFSv2 e NFSv3), o protocolo é stateless: o servidor não mantém nenhum registro de quais arquivos cada cliente tem abertos em um dado momento — cada requisição enviada pelo cliente carrega toda a informação necessária para ser atendida de forma independente das requisições anteriores. Essa escolha simplifica enormemente a recuperação após uma falha do servidor: como não existe estado de sessão para reconstruir, o servidor pode reiniciar e voltar a responder normalmente, sem executar nenhum protocolo especial de reconciliação com os clientes. A segunda característica é o uso de file handles: em vez de identificar um arquivo pelo seu caminho textual a cada operação, o NFS usa uma referência opaca e compacta — tipicamente composta por identificador do dispositivo, número do inode e um número de geração — obtida uma única vez, no momento da montagem, e reaproveitada em todas as operações seguintes sobre aquele arquivo.

A contrapartida dessas escolhas de projeto é justamente a semântica de consistência mencionada na seção anterior: o NFS clássico adota uma semântica de sessão, na qual as modificações feitas por um cliente em um arquivo só se tornam visíveis para outros clientes depois que o arquivo é fechado, e não imediatamente a cada escrita. Versões mais recentes do protocolo, como o NFSv4, tornam o protocolo stateful — introduzindo conceitos como leases e locks — justamente para oferecer garantias de consistência mais próximas da semântica Unix, ao custo de um servidor mais complexo.

Estudo de caso moderno: o Google File System (GFS)

Se o NFS representa o DFS clássico voltado a compartilhar arquivos de uso geral entre estações de trabalho, o Google File System (GFS), descrito por Ghemawat, Gobioff e Leung em um artigo de 2003, representa uma geração posterior de sistemas de arquivos distribuídos, projetados para um cenário bem diferente: armazenar arquivos muito grandes, com padrão de acesso predominantemente sequencial — leituras que percorrem o arquivo inteiro e escritas que, na maior parte das vezes, apenas adicionam dados ao final dele (append) — rodando sobre milhares de máquinas comuns, de baixo custo, em que falhas de hardware são consideradas a norma esperada, e não uma exceção rara a ser tolerada apenas ocasionalmente.

A arquitetura do GFS reflete diretamente esse cenário. Cada arquivo é dividido em fragmentos de tamanho fixo, chamados chunks, de 64 megabytes cada. Um único processo master mantém todos os metadados do sistema — o espaço de nomes de arquivos e diretórios, o mapeamento de cada arquivo para a lista de chunks que o compõe, e a localização de cada chunk entre os servidores disponíveis — enquanto o armazenamento físico dos dados propriamente ditos fica a cargo de um grande número de chunkservers, cada um guardando chunks em seu disco local. Por padrão, cada chunk é replicado em três chunkservers diferentes, o que garante que a perda de uma máquina não implique a perda de dados.

O detalhe mais importante dessa arquitetura, do ponto de vista de desempenho, é que o master nunca fica no caminho dos dados. Quando um cliente quer ler ou escrever um arquivo, ele primeiro contata o master apenas para descobrir em quais chunkservers estão os chunks relevantes; a partir daí, toda a transferência de dados propriamente dita acontece diretamente entre o cliente e os chunkservers envolvidos, sem passar pelo master. Como o volume de metadados é muito menor que o volume de dados, essa separação evita que o master se torne um gargalo, mesmo com um número muito grande de clientes acessando o sistema simultaneamente.

Do GFS ao HDFS

O artigo original do GFS teve um impacto que vai muito além do uso interno do Google: ele influenciou diretamente o projeto do HDFS (Hadoop Distributed File System), a camada de armazenamento do ecossistema Hadoop, amplamente usada em processamento de grandes volumes de dados. O HDFS reaproveita a mesma divisão de papéis do GFS, apenas com outros nomes: o NameNode do HDFS corresponde ao master do GFS, concentrando os metadados e o mapeamento de arquivos para blocos; os DataNodes correspondem aos chunkservers, armazenando fisicamente os blocos de dados; e o próprio conceito de chunk aparece no HDFS como block, originalmente também de 64 megabytes, tamanho que evoluiu para 128 megabytes como padrão nas versões mais recentes do sistema.

Essa herança de projeto também aparece na filosofia de acesso aos dados: assim como o GFS foi otimizado para escrita por append e leitura sequencial de arquivos grandes, o HDFS segue o modelo write-once-read-many, no qual arquivos costumam ser escritos uma única vez e lidos repetidamente por tarefas de processamento em lote, e não editados no lugar como um arquivo de uso pessoal. O fato de duas gerações de sistemas de arquivos distribuídos amplamente usados na indústria — um interno ao Google, outro aberto e usado por incontáveis empresas — convergirem para a mesma divisão fundamental entre um nó de metadados e vários nós de dados é uma evidência de quão bem essa arquitetura resolve o problema de escalar armazenamento distribuído para volumes muito grandes de dados.

Caching no cliente e o problema da coerência

Independentemente do DFS específico, uma técnica quase universal para melhorar o desempenho percebido pelo cliente é manter uma cópia em cache local dos dados acessados recentemente. Ler um arquivo do cache local evita uma ida à rede a cada acesso, reduz a latência percebida pelo usuário e alivia a carga sobre o servidor, que não precisa reatender repetidamente o mesmo pedido. Por essas razões, caching no cliente é considerado a técnica central de desempenho em sistemas de arquivos distribuídos — praticamente todo DFS de uso geral, do NFS em diante, incorpora alguma forma dela.

O problema é que cache introduz, de forma quase inevitável, uma questão de coerência: se o cliente A lê um arquivo e guarda uma cópia em seu cache local, e em seguida o cliente B modifica esse mesmo arquivo diretamente no servidor, a cópia que o cliente A mantém em cache fica desatualizada — ela não reflete mais o estado real do

arquivo. Esse é, em essência, o mesmo dilema da coerência de cache que aparece em arquitetura de hardware, quando vários processadores mantêm cópias da mesma linha de memória em seus caches locais e é preciso garantir que nenhum deles opere sobre um valor obsoleto. A diferença é de escala e de latência: em vez de processadores conectados por um barramento rápido dentro da mesma máquina, aqui são clientes espalhados por uma rede, com atrasos ordens de grandeza maiores e comunicação bem menos previsível.

Diferentes DFS resolvem esse problema de formas diferentes. Uma abordagem é a validação por timestamp, em que o cliente, antes de efetivamente usar uma cópia em cache, confirma com o servidor se ela ainda corresponde à versão mais recente do arquivo. Outra é a invalidação por callback, usada por exemplo no Andrew File System (AFS): o próprio servidor assume a responsabilidade de avisar proativamente os clientes que têm uma cópia em cache sempre que o arquivo correspondente é modificado, dispensando o cliente de perguntar repetidamente. Uma terceira abordagem, adotada pelo NFS clássico, é simplesmente adiar o problema por meio da semântica de sessão já discutida: como as modificações só se tornam visíveis a outros clientes depois que o arquivo é fechado, o sistema evita ter que lidar com o caso mais difícil, que é o de coerência entre clientes que mantêm o mesmo arquivo aberto e modificado ao mesmo tempo.

Próxima aula

Com a Aula 26 fechamos o bloco de sistemas operacionais distribuídos: partimos da arquitetura geral de um SOD (Aula 24), passamos pela gerência de processos e escalonamento distribuído (Aula 25) e chegamos aos sistemas de arquivos distribuídos, do NFS ao GFS e ao problema de coerência de cache (Aula 26). A Aula 27 muda de natureza: é uma aula de revisão geral do 2º bimestre, que retoma os principais conceitos vistos desde a Aula 14 — computação em grid, algoritmos distribuídos, tolerância a falhas e sistemas operacionais distribuídos — e dedica tempo a orientar os checkpoints e a entrega final do trabalho evolutivo da disciplina.

Referências

TANENBAUM, Andrew S.; VAN STEEN, Maarten. Distributed Systems: Principles and Paradigms (edição atual: Distributed Systems). Pearson. Capítulo sobre sistemas de arquivos distribuídos: requisitos, NFS e semântica de consistência.

COULOURIS, George et al. Sistemas Distribuídos: Conceitos e Projeto. 5ª ed. Porto Alegre: Bookman. Capítulo sobre serviços de arquivos distribuídos: transparência, estudos de caso e técnicas de caching.

GHEMAWAT, Sanjay; GOBIOFF, Howard; LEUNG, Shun-Tak. "The Google File System". In: Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP), 2003. Artigo original que descreve a arquitetura do GFS.